Optimization Techniques

Lab n°7 - Genetic Algorithms

The following pseudo-code illustrates a binary-encoded genetic algorithm with single-point crossover and tournament scheme for constructing the mating pool.

Algorithm 1 Binary-Encoded Genetic Algorithm

Input: *an initial population* $\mathcal{P} = \{\boldsymbol{x}_k\}_k$, *population size* N_{pop}, *number of generations* N_{step}, *mutation rate* p_{mut}, *a fitness function* fitness

$n \leftarrow \text{length}(\boldsymbol{x}_1)$ ▷ size of each chromosome
$f_{best} \leftarrow -\infty$ ▷ initialize the best-so-far
$\boldsymbol{x}_{best} \leftarrow \emptyset$
for $t = 1 \rightarrow N_{step}$ **do**
  Selection (Tournament): ▷ Creation of the mating pool
  $\mathcal{P}_{\text{pool}} \leftarrow \emptyset$
  for $i = 1 \rightarrow N_{\text{pop}}$ **do**
    $k_1, k_2 \leftarrow$ random indices from $\{1, \ldots, N_{\text{pop}}\}$
    $\boldsymbol{x}_{win} \leftarrow \text{argmax}\{\text{fitness}(\boldsymbol{x}_{k_1}), \text{fitness}(\boldsymbol{x}_{k_2})\}$
    $\mathcal{P}_{\text{pool}} \leftarrow \mathcal{P}_{\text{pool}} \cup \{\boldsymbol{x}_{win}\}$
  end for
  Crossover (Single-Point):
  $\mathcal{P}_{\text{offspring}} \leftarrow \emptyset$
  for $i = 1, 3, \ldots, N_{\text{pop}} - 1$ **do**
    $\text{cr} \leftarrow$ random index $\in \{1, \ldots, n-1\}$
    $\boldsymbol{c}_1 \leftarrow [\boldsymbol{x}_i(1:\text{cr}), \boldsymbol{x}_{i+1}(\text{cr}+1:n)]$
    $\boldsymbol{c}_2 \leftarrow [\boldsymbol{x}_{i+1}(1:\text{cr}), \boldsymbol{x}_i(\text{cr}+1:n)]$
    $\mathcal{P}_{\text{offspring}} \leftarrow \mathcal{P}_{\text{offspring}} \cup \{\boldsymbol{c}_1, \boldsymbol{c}_2\}$
  end for
  Mutation:
  for each $\boldsymbol{x} \in \mathcal{P}_{\text{offspring}}$ **do**
    for $j = 1 \rightarrow n$ **do**
      if $\text{rand}(0,1) \leq p_{mut}$ **then**
        $x_j \leftarrow 1 - x_j$ ▷ Flip bit
      end if
    end for
  end for
  $\mathcal{P} \leftarrow \mathcal{P}_{\text{offspring}}$
  Update Best Solution:
  $f_{curr}, idx \leftarrow \max_j\{\text{fitness}(\boldsymbol{x}_j) \mid \boldsymbol{x}_j \in \mathcal{P}\}$
  if $f_{curr} > f_{best}$ **then**
    $f_{best} \leftarrow f_{curr}$
    $\boldsymbol{x}_{best} \leftarrow \boldsymbol{x}_{idx}$
  end if
end for
return $f_{best}, \boldsymbol{x}_{best}$

Exercise 1 *Use a binary encoded genetic algorithm (possibly inspired by Algorithm 1) to solve the 0-1 Knapsack problem.*

The file `knapsack.csv` *provides a list of 100 items. The first column contains the weights, the second column the items.*

Set the maximum weight to 1965. Try with a mating pool of size 50 and a mating pool of size 500.

The optimum is attained at 4966 of total value. Test the convergence speed of the algorithm as the size of the mating pool varies.

Hint. Even starting with an initial population of admissible items combinations (i.e., all respect the weight limit), the crossover and the mutation could lead to non-admissible chromosomes.

Two strategies can be used:

1. Every non-admissible chromosome is refused, and the parents are copied in the mating pool again.
2. Instead of using as fitness the sum of values, one uses a penalized fitness of the form

$$fit(\boldsymbol{x}) = \sum_{i=1}^{n} x_i v_i - \alpha * \max(0, \sum_{i=1}^{n} x_i w_i - W),$$

where $\alpha = 10 \div 100$. In this way, if the chromosome is admissible, $\max(0, \sum_{i=1}^{n} x_i w_i - W) = 0$ and the fitness is equal to the total value of the items. If the chromosome is not admissible, its fitness is penalized proportionally to the exceeding weight.

Exercise 2 *Algorithm 2 represents the pseudo-code for a real-encoded genetic algorithm, with roulette-wheel selection and elitism (the best-so-far element is always copied in the mating pool).*

Use a real-encoded genetic algorithm (possibly inspired from Algorithm 2) to find the minimum of the function

$$f(\boldsymbol{x}) = x_1^4 + x_2^4 - 4x_1^3 - 3x_2^3 + 2x_1^2 + 2x_1 x_2.$$

1. *First, you can start with a population of* $N = 100$ *elements in the square* $[-10, 10] \times [-10, 10]$.
2. *You can subsequently refine the algorithm by restricting the initial square.*

The global minimum is $\boldsymbol{x}^* = (2.67321, -0.675885)$, minimum cost $f(\boldsymbol{x}^* = -13.5320)$.

Hint: to visualize the evolution of the population, you can use the Matlab command "pause":

```
plot(xpop(:,1),xpop(:,2), '.')
pause(0.5) % 0.5 seconds
```

In Python, this should be something like

```
import time
time.sleep(0.5) % 0.5 seconds
```

Algorithm 2 Genetic Algorithm for Real-Encoded Continuous Optimization

Input: *objective function f of n variables, population size N, mutation rate p_{mut}, generations n_{step}, bounds $[x_{min}, x_{max}]$.*

$L \leftarrow |x_{max} - x_{min}|$ ▷ Domain scale for mutation
if $N \pmod 2 \neq 0$ **then** $N \leftarrow N + 1$ ▷ Ensure even population for pairing
end if
$\mathbf{X} \leftarrow \{x_{min} + (x_{max} - x_{min}) \cdot \text{rand}(n)\}_{i=1\ldots N}$ ▷ Initial population
$f_{best} \leftarrow \infty$
for $k = 1 \rightarrow n_{step}$ **do**
    $\mathbf{\Phi} \leftarrow f(\mathbf{X})$ ▷ Evaluate objective function for all individuals
    $[\Phi_{min}, idx] \leftarrow \text{findmin}(\mathbf{\Phi})$
    if $\Phi_{min} < f_{best}$ **then** ▷ Update Global Best (Elitism Memory)
        $f_{best} \leftarrow \Phi_{min}$
        $x_{best} \leftarrow \mathbf{X}[idx]$
    end if
    $\phi_{high} \leftarrow \max(\mathbf{\Phi})$
    $\mathbf{w} \leftarrow \phi_{high} - \mathbf{\Phi}$ ▷ Mapping minimization to maximization weights
    $\mathbf{S} \leftarrow \text{cumsum}(\mathbf{w}/\sum \mathbf{w})$ ▷ Cumulative distribution for Roulette Wheel
    $\mathbf{X}_{sel} \leftarrow \emptyset$
    for $i = 1 \rightarrow N - 1$ **do** ▷ Select $N-1$ individuals via Roulette Wheel
        $r \leftarrow \text{rand}()$
        $j \leftarrow \text{findfirst}(s \in \mathbf{S} \mid s \geq r)$ ▷ Finds the smallest j such that $S(j) \geq r$
        $\mathbf{X}_{sel} \leftarrow [\mathbf{X}_{sel}, \mathbf{X}[j]]$
    end for
    $\mathbf{X}_{sel} \leftarrow [\mathbf{X}_{sel}, \text{copy}(x_{best})]$ ▷ Deterministic Elitism: add the best-so-far
    $\mathbf{X}_{new} \leftarrow \emptyset$
    for $i = 1, 3, \ldots, N - 1$ **do** ▷ Reproduction and Crossover
        $p_1, p_2 \leftarrow \mathbf{X}_{sel}[i], \mathbf{X}_{sel}[i+1]$
        $\lambda \leftarrow \text{rand}()$
        $c_1 \leftarrow \lambda p_2 + (1 - \lambda) p_1$ ▷ Arithmetic Crossover
        $c_2 \leftarrow \lambda p_1 + (1 - \lambda) p_2$
        for $c \in \{c_1, c_2\}$ **do** ▷ Mutation process
            if $\text{rand}() \leq p_{mut}$ **then**
                $c \leftarrow c + L \cdot 0.05 \cdot (\text{rand}(n) - 0.5)$ ▷ Local disturbance
            end if
            $\mathbf{X}_{new} \leftarrow [\mathbf{X}_{new}, c]$
        end for
    end for
    $\mathbf{X} \leftarrow \mathbf{X}_{new}$ ▷ Generational replacement
end for
return f_{best}, x_{best}